Efficient Search in a 2D Matrix with Sorted Rows and Columns

Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties:

- Integers in each row are sorted in ascending order from left to right.
- Integers in each column are sorted in ascending order from top to bottom.

Examples:

- **Example 1:**
    - **Input:**

matrix = [
  [1, 4, 7, 11, 15],
  [2, 5, 8, 12, 19],
  [3, 6, 9, 16, 22],
  [10, 13, 14, 17, 24],
  [18, 21, 23, 26, 30]
], target = 5

- **Output:** true
- **Explanation:** The number 5 is present in the matrix.

Example 2:

- **Input:**

matrix = [
  [1, 4, 7, 11, 15],
  [2, 5, 8, 12, 19],
  [3, 6, 9, 16, 22],
  [10, 13, 14, 17, 24],
  [18, 21, 23, 26, 30]
], target = 20

- **Output:** false
- **Explanation:** The number 20 is not present in the matrix.

Constraints

- m == matrix.length
- n == matrix[i].length
- 1 <= n, m <= 300
- -10^9 <= matrix[i][j] <= 10^9
- All the integers in each row are sorted in ascending order.
- All the integers in each column are sorted in ascending order.
- -10^9 <= target <= 10^9

Dynamic Product Suggestions Based on Incremental Search Input

You are given an array of strings products and a string searchWord. Design a system that suggests at most three product names from products after each character of searchWord is typed. Suggested products should have a common prefix with searchWord. If there are more than three products with a common prefix, return the three lexicographically minimum products.

Return a list of lists of the suggested products after each character of searchWord is typed.

Examples

- **Example 1:**
  - **Input:** products = ["mobile", "mouse", "moneypot", "monitor", "mousepad"], searchWord = "mouse"
  - **Output:** [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]
  - **Explanation:** Products sorted lexicographically = ["mobile","moneypot","monitor","mouse","mousepad"]. After typing 'm' and 'mo', all products match and we show the user ["mobile","moneypot","monitor"]. After typing 'mou', 'mous', and 'mouse', the system suggests ["mouse","mousepad"].
- **Example 2:**
  - **Input:** products = ["havana"], searchWord = "havana"
  - **Output:** [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
  - **Explanation:** The only word "havana" will always be suggested while typing the search word.

Constraints

- 1 <= products.length <= 1000
- 1 <= products[i].length <= 3000
- 1 <= sum(products[i].length) <= 2 * 10^4
- All the strings of products are unique.
- products[i] consists of lowercase English letters.
- 1 <= searchWord.length <= 1000
- searchWord consists of lowercase English letters.

Finding or Inserting a Target Score in a Sorted List of User Scores

You are developing a search feature for an application that maintains a sorted list of unique integers representing user scores.

Given a sorted array of distinct integers nums and an integer target, your task is to write a function that searches for the target in nums. If the target exists, return its index. If it does not exist, return the index where it would be inserted to maintain the sorted order.

To achieve this, your algorithm must have a runtime complexity of O(log n).

Examples

- **Example 1:**
  - **Input:** nums = [1, 3, 5, 6], target = 5
  - **Output:** 2
  - **Explanation:** 5 exists in nums at index 2.
- **Example 2:**
  - **Input:** nums = [1, 3, 5, 6], target = 2
  - **Output:** 1
  - **Explanation:** 2 does not exist in nums, but it would be inserted at index 1.
- **Example 3:**
  - **Input:** nums = [1, 3, 5, 6], target = 7
  - **Output:** 4
  - **Explanation:** 7 does not exist in nums, but it would be inserted at index 4.

Constraints

- 1 <= nums.length <= 10^4
- -10^4 <= nums[i] <= 10^4
- nums contains distinct values sorted in ascending order.
- -10^4 <= target <= 10^4

Reordering Linked List by Odd and Even Indices

Imagine you're developing a simulation of a traffic system for self-driving cars. In this system, every vehicle's position on the road is tracked at regular intervals. However, to optimize the traffic management process, the positions of the vehicles must be grouped based on their recorded indices — cars at odd indices should be tracked first, followed by cars at even indices. This mirrors the problem of reordering nodes in a singly linked list by grouping nodes at odd indices together followed by nodes at even indices.

Your task is to write a function that rearranges the linked list nodes based on this criterion. The first node is considered "odd," the second node "even," and so on. You must ensure that the relative order of the nodes in both odd and even groups remains unchanged, and the solution must be done in O(1) extra space complexity and O(n) time complexity.

Test Cases

Test Case	Input List	Expected Output List
1	[1, 2, 3, 4, 5]	[1, 3, 5, 2, 4]
2	[2, 1, 3, 5, 6, 4, 7]	[2, 3, 6, 7, 1, 5, 4]
3	[4, 7]	[4, 7]
4	[1]	[1]
5	[]	[]

Checking if a Singly Linked List is a Palindrome

You need to determine whether a singly linked list is a palindrome. A linked list is considered a palindrome if the values read from the front to the back are the same as when read from the back to the front.

Problem Statement:
Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Constraints:

- The number of nodes in the list is in the range [1,105][1, 10^5][1,105].
- 0≤Node.val≤90

Test Cases:

Test Case #	Input	Output
1	[1, 2, 2, 1]	true
2	[1, 2]	false
3	[1]	true
4	[9, 8, 9]	true
5	[1, 0, 1, 0, 1]	false

Deleting a Node from a Singly Linked List

You are required to delete a specific node from a singly linked list. You will not have access to the head of the list, but you will be given the node that needs to be deleted. The task is to ensure that after deletion, the order of the remaining nodes in the list is preserved.

Test Cases

Test Case	Input Linked List	Node to be Deleted	Expected Output
1	[4, 5, 1, 9]	5	[4, 1, 9]
2	[4, 5, 1, 9]	1	[4, 5, 9]
3	[1, 2, 3, 4]	2	[1, 3, 4]
4	[1, 2]	1	[2]
5	[10, 20, 30]	20	[10, 30]

Constraints

- The number of nodes in the given linked list is in the range [2,1000][2, 1000][2,1000].
- The value of each node in the list is unique.
- The node to be deleted is guaranteed to be in the list and is not a tail node.
- −1000≤Node.val≤1000-1000 \leq \text{Node.val} \leq 1000−1000≤Node.val≤1000.

Removing Outermost Parentheses: Simplifying Valid Strings

A valid parentheses string is either empty "", "(" + A + ")", or A + B, where A and B are valid parentheses strings, and + represents string concatenation.

- A valid parentheses string s is **primitive** if it is nonempty and there does not exist a way to split it into s = A + B, with A and B nonempty valid parentheses strings.
- Given a valid parentheses string s, consider its primitive decomposition: s = P1 + P2 + ... + Pk, where Pi are primitive valid parentheses strings.

The task is to return the string s after removing the outermost parentheses of every primitive string in the primitive decomposition of s.

Examples:

Example 1:

- **Input:** s = "((()))(())"
- **Output:** "()()()"
- **Explanation:** The input string is "((()))(())", with primitive decomposition "((()))" + "(())". After removing outer parentheses of each part, this is "()()" + "()" = "()()()".

Example 2:

- **Input:** s = "((()))(()))(()(()))"
- **Output:** "()()()()(())"
- **Explanation:** The input string is "((()))(()))(()(()))", with primitive decomposition "((()))" + "(())" + "((()(")". After removing outer parentheses of each part, this is "()()" + "()" + "()(()" = "()()()()(())".

Example 3:

- **Input:** s = "()()"
- **Output:** ""
- **Explanation:** The input string is "()()", with primitive decomposition "()" + "()". After removing outer parentheses of each part, this is "" + "" = "".

Constraints:

- 1 <= s.length <= 10^5
- s[i] is either '(' or ')'.
- s is a valid parentheses string.

Calculating Scores in a Baseball Game with Custom Rules

You are keeping the scores for a baseball game with strange rules. The game starts with an empty record, and you're given a list of operations to apply on the record. The operations can be one of the following:

1. An integer x: Record a new score of x.
2. "+": Record a new score that is the sum of the previous two scores.
3. "D": Record a new score that is the double of the previous score.
4. "C": Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the operations.

Example:

Example 1:

- **Input:** Enter the number of operations :
- **Input:** ops = ["5", "2", "C", "D", "+"]
- **Output:** 30
- **Explanation:**
    - "5": Add 5 to the record, now [5].
    - "2": Add 2 to the record, now [5, 2].
    - "C": Invalidate and remove the previous score, now [5].
    - "D": Double the previous score (5), now [5, 10].
    - "+": Sum of the previous two scores (5 + 10), now [5, 10, 15].
    - Total sum: 5 + 10 + 15 = 30.

Example 2:

- **Input:** ops = ["5", "-2", "4", "C", "D", "9", "+", "+"]
- **Output:** 27
- **Explanation:**
    - "5": Add 5 to the record, now [5].
    - "-2": Add -2 to the record, now [5, -2].
    - "4": Add 4 to the record, now [5, -2, 4].
    - "C": Invalidate and remove the previous score, now [5, -2].
    - "D": Double the previous score (-2), now [5, -2, -4].
    - "9": Add 9 to the record, now [5, -2, -4, 9].
    - "+": Sum of the previous two scores (-4 + 9), now [5, -2, -4, 9, 5].
    - "+": Sum of the previous two scores (9 + 5), now [5, -2, -4, 9, 5, 14].
    - Total sum: 5 + (-2) + (-4) + 9 + 5 + 14 = 27.

Example 3:

- **Input:** ops = ["1", "C"]
- **Output:** 0
- **Explanation:** "1": Add 1 to the record, then "C" invalidates it. The total sum is 0.

Constraints:

- 1 <= operations.length <= 1000
- Each operation is "C", "D", "+", or a string representing an integer in the range [-30,000, 30,000].
- For operation "+", there will always be at least two previous scores on the record.
- For operations "C" and "D", there will always be at least one previous score on the record.